

Título del informe

Tema a tratar

Integrantes: Integrante 1
Integrante 2
Profesor: Profesor 1
Auxiliar: Auxiliar 1
Ayudantes: Ayudante 1
Ayudante 2
Ayudante de laboratorio: Ayudante 1

Fecha de realización: 30 de julio de 2026
Fecha de entrega: 30 de julio de 2026
Santiago de Chile

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice de Contenidos

1. Cálculo Numérico	1
1.1. Errores Computacionales	1
1.2. Derivadas numéricas	2
2. Breve introducción a Python	4
3. Introducción a la Búsqueda de Ceros	5

Índice de Figuras

1. Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h	3
---	---

Índice de Códigos

1. Suma de dos decimales en Python	1
2. Output	1

1. Cálculo Numérico

1.1. Errores Computacionales

Si usted ha intentado realizar una suma de dos números decimales mediante algún lenguaje de programación, podrá haber notado que el resultado no es exactamente correcto

Código 1: Suma de dos decimales en Python

```
1 a = 0.1
2 b = 0.2
3 c = 0.3
4
5 print(f"{a + b == c}")
6 print(f"{a + b}")
```

Cuyo output será

Código 2: Output

```
1 False
2 0.30000000000000004
```

Este error se debe a la manera en que las computadoras deben representar números. En general, los humanos representamos los números en base 10, por ejemplo, escribimos el 325 como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 = [325]_{10}$$

Pero los computadores utilizan bits (0 y 1), por lo que estos representan números en base 2, es decir:

$$325 = 2^8 + 2^6 + 2^2 + 2^0 = [101000101]_2$$

Ahora, para representar número reales en el sistema binario, se deben incluir exponentes negativos a la suma de exponentes. En la representación binaria, los números se separan por un exponente y una parte fraccionaria (mantisa) las cuales son separadas por un punto (.). Por ejemplo,

$$\begin{aligned}[2.145]_{10} &= 2 \cdot 10^0 + \frac{1}{8} \\ &= 2^1 + 2^{-3} \\ &= [10.001]_2 \\ &= [1.0001]_2 \cdot 2^1\end{aligned}$$

Este ejemplo en particular funciona bien porque la mantisa del número decimal resulta ser un exponente de 2, veamos que ocurre para un número más complejo:

$$\begin{aligned}
[0.1]_{10} &= [1]_2/[1010]_2 \\
&= [0.00011]_2 \\
&= [0.00011001100110011 \dots] \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots \\
&= 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots)
\end{aligned}$$

Obtenemos una serie infinita de potencias de dos. Como en la practica los computadores tienen memoria finita es necesario truncar la serie. En general, la mantisa recibe 52 bits, por lo que la serie se trunca hasta el termino 2^{-52} . en efecto:

$$[0.1]_{10} = 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots + 2^{-52})$$

Este número no será *exactamente* igual a 0.1, si no que será igual a $0.1 \cdot (1 + \epsilon)$. Este será el caso para la gran mayoría de los calculos hechos computacionalmente, el resultado final tendrá una pequeña divergencia con respecto al valor real, es decir

$$\bar{a} = a \cdot (1 + \epsilon)$$

donde $\bar{a}$ y a son los valores calculado y real, respectivamente. La diferencia entre el número deseado y el resultante se denomina **error computacional**. Dado que las computadores no pueden físicamente lidiar con cantidades infinitesimalmente pequeñas, muchos de los métodos de cálculo adaptados a la programación que veremos a lo largo del apunte serán basados en aproximaciones, por esta razón es importante aprender a identificar fuentes de errores numericos y sus ordenes de magnitud.

1.2. Derivadas numéricas

Sabemos del calculo diferencial que la derivada de una función arbitraria f evaluada en un punto arbitrario x_0 es dada por

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h} \quad (1)$$

pero ya sabemos que necesitamos que h sea un número finito, por lo que estamos obligados a seguir anaizando esta definición. De la serie de taylor de f evaluada en un punto $a + h$, $h > 0$, tenemos que:

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(a)h^3 + \dots \quad (2)$$

Podemos reemplazar los terminos de orden superior por un $f(\xi)h^2$, $a \leq \xi \leq a + h$ (Teorema del valor medio), luego

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (3)$$

$$f'(a) = \frac{f(a + h) - f(a)}{h} - \frac{1}{2!}f''(\xi)h \quad (4)$$

Aquí, hemos obtenido una expresión finita para la derivada de una función evaluada en un punto, denominada *derivada adelantada*. Ahora, si buscamos el error de la ecuación (4) vemos que

$$Error = \left| f'(a) - \frac{f(a + h) - f(a)}{h} \right| = \frac{1}{2!} |f''(\xi)| h \quad (5)$$

en notación $\mathcal{O}$ de Landau,

$$Error = \frac{1}{2!} |f''(\xi)| h = \mathcal{O}(h) \quad (6)$$

Esto quiere decir que el error de truncamiento de la derivada adelantada será del orden de h , como se puede observar en la figura 1, el error de la derivada decrece linealmente con el orden de magnitud de h hasta aproximadamente $h \sim 10^{-8}$, pasado ese punto h se vuelve demasiado pequeño para la computadora y predomina el error computacional. Por esta razón, a la hora de implementar este, o cualquiera de los métodos expuestos en este apunte, es conveniente utilizar un $h \sim 10^{-5}$.

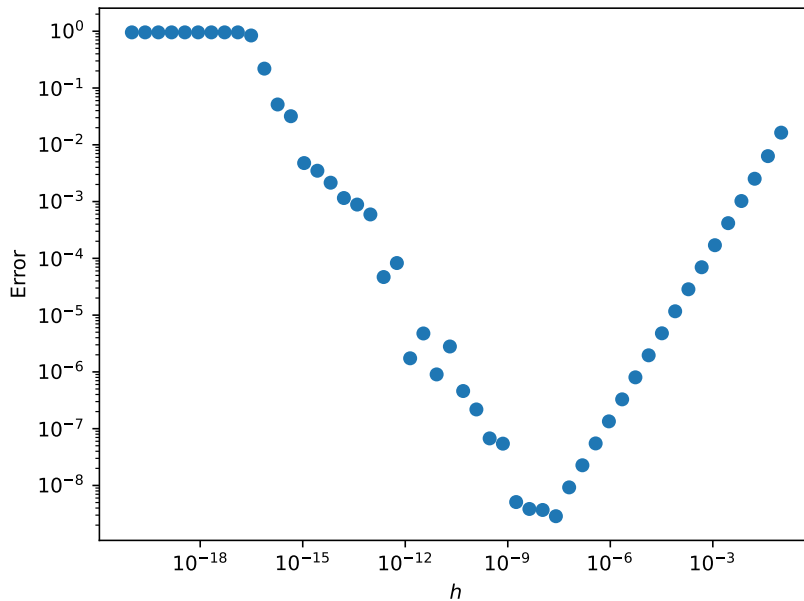


Figura 1: Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h

2. Breve introducción a Python

Hola

3. Introducción a la Búsqueda de Ceros

Usualmente, cuando trabajamos con ecuaciones, una de nuestras metas es encontrar el valor de cierta variable o también llamada incógnita, para esto, desarrollamos una serie de pasos o "algoritmo" que seguir para encontrar la respuesta. Un buen ejemplo de esto es considerar el caso de una ecuación cuadrática:

$$ax^2 + bx + c = 0 \quad (7)$$

Es bien sabido que, dado los coeficientes a, b y c, existe una fórmula general que seguir, la conocida como ecuación cuadrática o *Ecuación de Bashkara*:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (8)$$

Al introducir los respectivos valores de los coeficientes, podemos encontrar las dos soluciones a este problema.

Sin embargo, no siempre se puede encontrar una ecuación para los valores de incógnitas en toda ecuación, un ejemplo a esto es el siguiente:

$$\sin x + x = 0 \quad (9)$$

A pesar de lo simple de nuestro problema (puesto que es la suma entre una función trigonométrica y la incógnita), ya no es posible encontrar una ecuación de forma analítica que exprese valores exactos de las soluciones para "x", así, la pregunta es ¿Qué podemos hacer en estos casos?

Para esto, se desarrollan los **Métodos para hallar ceros de una función**

meow